CHAPTER 13

Testing for Security

Web security is a scary topic. All of your applications depend on cryptography and code that is beyond your immediate control. Despite that, certain parts of web security *are* in your control—all the logins and access checks and injection errors that happen on your site as a result of programming choices you make.

When it comes to security and testing, there's good news and bad news. The good news is that all kinds of access and injection bugs are amenable to automated developer testing. Sometimes unit testing will do the trick; other times end-to-end testing is the correct tool, but the effects of a security problem are often easily reproducible in a test environment. The bad news is that you need to actively determine where access and injection bugs might lurk in your code. This chapter focuses on user logins, roles, and using tests to make sure basic user authentication holds in your application.

Prescription 29	Security issues are often just bugs. Most of the practices you follow to keep your code bug-free will also help prevent and diagnose security issues.

User Authentication and Authorization

You've gotten quite far in your example without adding a user model to it, which we'll rectify now.

We need to get users and passwords into the system without spending too much time in the setup weeds—so you can focus your attention on the security issues that having users causes. To do that, use the Devise gem for basic user authentication and focus on how to use Devise as part of your security